

Veloce

Post-Quantum Cryptography SDK

V1 Engineering Plan

Organization	Lightrider Inc
Product	Veloce: cryptographic search SDK + FIPS entropy generator
Version	0.3 (vendor-response aligned)
Date	August 27, 2026
Prepared for	Martin Guo, Quantum Applied Scientist
Status	Linux reference implementation complete; Windows FIPS track unblocked by wolfSSL confirmations of 202

Abstract. Veloce V1 delivers two products in one SDK: **qSearch**, a high-performance cryptographic discovery tool (source, dependencies, certificates, network) that produces a Cryptography Bill of Materials (CBOM); and the **Veloce crypto core**, a locally deployed entropy generator and PQC service built on the wolfCrypt FIPS 140-3 module (certificate #4718) seeded by the Lightrider local entropy provider: a custom generate-seed callback drawing OS kernel entropy, verified in-process by Lightrider SP 800-90B-style health tests, fail-closed. The module makes no entropy claim and falls under legacy IG 9.3.A (vendor-confirmed 2026-08-27), so the external documented seed source is the compliant design. The SDK exposes ML-KEM, ML-DSA, and hybrid TLS 1.3 through a Python SDK and a desktop UI. Cloud connectivity to the Lightrider EMS is optional, off by default, and includes a user-controllable cloud-entropy mix-in that never weakens the local FIPS entropy claim. The checked-in implementation is currently Linux x86-64; the Windows FIPS DLL build path is vendor-confirmed and awaits only the published #4718 OE update for the validated deployment claim.

Contents

1	Executive Summary	3
2	Scope and Requirements	5
2.1	In scope (V1)	5
2.2	Out of scope (V1)	5
2.3	Standards and policy baseline	6
3	System Architecture	6
3.1	Component responsibilities	7
4	Pillar A: qSearch, Cryptographic Search	7
4.1	Collector adapters (replaceable)	7
4.2	Lightrider-owned layers	8
4.3	One internal finding format	8
4.4	Detection targets and outputs	8
4.5	Coverage assurance and outlier names	8
4.6	Optional AI triage	9
4.7	Performance approach	9
5	Pillar B: Entropy Generator and FIPS Crypto Core	9
5.1	Entropy pipeline	10
5.2	FIPS module integration	11
5.3	PQC provider outside the FIPS boundary	11
5.4	Hybrid TLS 1.3	12
5.5	Performance plan	12
6	Pillar C: EMS Cloud Integration	12
6.1	Modes and configuration	12
6.2	User-controllable cloud entropy (V1 addition)	13
6.3	Desktop UI: entropy specs and cloud-EMS control (V1 addition)	13
7	APIs and Deliverables	14
7.1	Public Python API (17 functions, fixed by the guidance)	14
7.2	Deliverables checklist	14
7.3	Client installation experience	14
7.4	Platform-separated development and release layout	15
7.5	Startup branding	15
8	Security and Compliance	15
8.1	Licensing and distribution model	16
9	Test Gates and Acceptance	17
10	Execution Guide: What To Do, In Order	17
10.1	Component-to-repository traceability	18
11	Risks and Open Items	18
A	Build Recipes	19
B	Manual Two-Host Interoperability Test	20
C	CBOM Field Mapping (SDK → workbook)	20

D References

20

1 Executive Summary

Veloce is Lightrider's V1 PQC SDK. It comprises a cryptographic discovery component (qSearch) and a FIPS 140-3 crypto core with a validated entropy pipeline. Both operate fully offline; an optional EMS connection provides fleet telemetry and user-controlled cloud entropy.

Document organization. This document is the implementation specification for Veloce V1 and is sufficient for reimplementing on its own. Sections 3–6 define the components, Section 7 the public API and deliverables, Section 8 the security controls, Section 9 the acceptance tests, Section 10 the build order with a component-to-repository map, and the appendices the exact build commands.

Implementation snapshot (2026-08-27). The checked-in implementation contains the C++ agent, pure-Python client, Rust CLI, native Rust qSearch source/system scanner, CBOM outputs, the Veloce Desktop application (local web UI: security dashboard + click-to-scan qSearch), Windows/macOS desktop build paths, and a 42-test gate battery. The current source scanner is a single-threaded recursive literal-pattern engine, not yet the parallel collector design described as the target below. Hybrid-TLS policy selection exists, but the TLS data plane, native Windows FIPS runtime validation, dependency/PCAP collectors, and the EMS network client are not implemented. This distinction is normative: target requirements must not be reported as delivered functionality.

Vendor response snapshot (2026-08-27). wolfSSL confirmed in writing: (1) Windows FIPS supports *DLL builds only* (the bundled README's static-library statement is a legacy error, correction in wolfSSL PR #11142); build configurations are `DLL Debug | x64 / DLL Release | x64` in `WIN10/wolfssl-fips.sln`. (2) The exact `v5.2.1 user_settings.h` is supplied (`HAVE_FIPS_VERSION_MAJOR 5, MINOR 2, PATCH 1`). (3) CAVP/CMVP testing for the Windows 11 Pro / Intel i7-1260P (Lenovo ThinkPad T14) OE and an Azure Linux OE (Ubuntu 24.04 on Docker 28.5 / Hyper-V, Xeon Platinum 8272CL) is *completed, awaiting publication* on certificate #4718 and its security policy. (4) The module makes no entropy claim; the CO registers a custom generate-seed callback via `wc_SetSeed_Cb()` (wolfSSL FIPS FAQ §2), and the module falls under *legacy* IG 9.3.A (validated before 2026-01-01), exempt from the current ESV verbiage. (5) *wolfEntropy was never tested with v5.2.1* and wolfSSL recommends not using it with the legacy module; `ESV #E335` belongs to the separate `v6.0.0` module, and the `v5.2.4` upgrade path is roughly a year out as a new certificate. (6) The wolfCrypt FIPS User Guide / COGM is now on hand and governs allowed build settings. These answers close the S0 vendor items; the only remaining external dependency for the Windows validated-deployment claim is CMVP publication of the tested OE rows.

The two pillars

Pillar A: qSearch (cryptographic search).

A Rust-based discovery engine with a native literal-pattern and PEM-certificate collector today, with replaceable dependency and network collectors planned. It finds RSA, DH, ECDH, ECDSA and other quantum-vulnerable indicators in source code and certificates, and emits JSON, CSV, CycloneDX CBOM, and OMB M-23-02 inventory fields. Lightrider owns the normalization, correlation, classification, CBOM generation, risk scoring, and reporting layers.

Pillar B: Entropy generator and FIPS crypto core.

A native C++17 agent embedding the wolfCrypt FIPS 140-3 validated module (CMVP cert #4718) whose SP 800-90A Hash_DRBG is seeded only by the Lightrider local entropy provider: a custom generate-seed callback (`wc_SetSeed_Cb()`, per wolfSSL FIPS FAQ §2) drawing CPU RDSEED hardware entropy (x86-64) and applying SP 800-90B RCT/APT health tests to every seed block in-process, fail-closed. No ESV certificate is claimed; the module makes no entropy claim and legacy IG 9.3.A applies (vendor-confirmed). The agent provides ML-KEM-768 key establishment and ML-DSA signatures, consumed from a pure-Python SDK over an authenticated local IPC channel. Hybrid TLS 1.3 policy selection is present; the sample handshake/data plane remains a G3 deliverable.

Technical stack

The product is designated **Veloce**; its components are qSearch, the Veloce Agent, the Veloce Python SDK, the Veloce CLI, and EMS (`light-rider-platform`).

Layer	Stack element	Technical basis
Crypto module	wolfCrypt FIPS 140-3 module v5.2.1, CMVP cert #4718, dynamically loaded	The guidance mandates a shared library/DLL (no static linking) and the purchased bundle carries this exact validated module; autotools (<code>--enable-fips=v5</code>) is its only FIPS-capable build path.
Entropy source	Lightrider local entropy provider: custom generate-seed callback wired via <code>wc_SetSeed_Cb()</code> , CPU RDSEED hardware entropy + SP 800-90B RCT/APT health tests, fail-closed; OS DRBG output only by explicit configuration, reported as an unvalidated SP 800-90C chain	The FIPS module itself makes no entropy claim and requires an external seed source (#4718 SP §11.1); under legacy IG 9.3.A (vendor-confirmed 2026-08-27) a documented external source is the compliant design. wolfEntropy is <i>not used</i> : wolfSSL confirmed it was never tested with v5.2.1 and recommends against it; ESV #E335 covers only the v6.0.0 module. Verification is engineering assurance, not an ESV credit. V1 entropy is software-only; hardware QRNG (IDQ Quantis) and the Quantropi quantum-entropy service are post-V1 integrations.
PQC provider	ML-KEM / ML-DSA compiled <i>beside</i> the FIPS boundary	The v5 FIPS build strips SHAKE/XOF from in-boundary SHA-3, so FIPS 203/204 code cannot link inside it; reported as <code>pqc_inside_fips_boundary: false</code> until wolfSSL's FIPS v7 "PQ-FS" module ships (Section 5.3).
Agent	C++17 native service	The Windows instruction set specifies C++17; wolfCrypt is a C library, making C++ the lowest-friction FFI; all keys and FIPS state must reside in one local process.
qSearch	Rust	The current standard-library-only implementation recursively scans UTF-8 files with literal patterns and inspects PEM OIDs; the target engine adds a compiled multi-pattern matcher and parallel walking. It links no wolfCrypt code.
SDK client	Pure Python over local IPC	The guidance mandates a pure-Python IPC client that never loads the crypto library; private keys are represented only as opaque handles.
Cloud entropy	EMS-delivered packets as DRBG <i>additional input</i> , default off	SP 800-90A permits additional input with zero credited entropy, so cloud entropy can strengthen output without altering the FIPS claims; the source documents prohibit entropy leaving the device, and this design sends none.
Platforms	Linux x86-64 implemented; Windows 11 x86-64 next (FIPS DLL configuration vendor-confirmed; OE publication pending); macOS separate track	Platform adapters and release artifacts remain isolated under one repository; no unimplemented platform may be advertised as supported.
Licensing	Lightrider commercial EULA wrapping the wolfSSL commercial license	The agreement (v.01-24) permits object-code redistribution of the Combined Program with notices retained (Section 8.1).

Terminology

Term	Meaning in this document
PQC	Post-quantum cryptography: algorithms that resist attack by quantum computers.
Quantum-vulnerable	Cryptography a large quantum computer could break: RSA, DH, ECDH, ECDSA.
ML-KEM / ML-DSA	The NIST-standardized post-quantum key-establishment (FIPS 203) and signature (FIPS 204) algorithms.

Hybrid TLS	A TLS 1.3 handshake combining a classical and a post-quantum key exchange; secure if either component holds.
FIPS 140-3	The U.S. government standard for validating cryptographic modules; validation is issued as a numbered CMVP certificate.
Module boundary	The exact code covered by a FIPS certificate; anything outside it carries no validation claim.
OE	Operational environment: the specific operating system and hardware a validation was tested on.
Entropy	Unpredictable data used to seed random-number generation; the root of all key security.
ESV	NIST Entropy Source Validation: certifies an entropy source under SP 800-90B.
DRBG	Deterministic random bit generator: expands a small entropy seed into unlimited random output (SP 800-90A).
RCT / APT	The two continuous entropy health tests (repetition count, adaptive proportion) required by SP 800-90B.
KAT / CAST	Known-answer test / conditional algorithm self-test: fixed-input checks a FIPS module runs on its own algorithms.
CBOM / SBOM	Cryptography / software bill of materials: an inventory of where and how cryptography (or software) is used.
Collector	A replaceable tool qSearch uses to gather raw evidence (source scan, certificate scan, network capture).
Correlate	Merge findings from different collectors that refer to the same asset.
Remediation	Changing or replacing vulnerable cryptography; V1 discovers and reports only.
IPC	Inter-process communication: the authenticated local channel between the Python SDK/CLI and the agent.
EMS	Lightrider's cloud platform (light-rider-platform) for fleet telemetry and cloud entropy.
M-23-02	The U.S. federal memorandum requiring agencies to inventory quantum-vulnerable cryptography.

2 Scope and Requirements

Scope is fixed by the PQC SDK guidance documents and the qSearch architecture notes. The single V1 scope change approved by Lightrider is the cloud-entropy mix-in (Section 6).

2.1 In scope (V1)

Area	V1 capability
qSearch discovery	Source-code and dependency scanning; certificate and protocol discovery; passive network/PCAP analysis; identification of RSA, DH, ECDH, ECDSA and other quantum-vulnerable cryptography
CBOM	Automatic CBOM export in JSON, CSV, and CycloneDX; M-23-02 inventory fields; executive summary; mapping to the Light Rider CBOM workbook
Entropy	Lightrider local entropy provider (CPU RDSEED hardware entropy via custom generate-seed callback), SP 800-90B RCT/APT health tests on every seed block, fail-closed seeding/reseeding of the wolfCrypt DRBG; live entropy specs surfaced in the SDK, CLI, and desktop UI
PQC services	ML-KEM key generation / encapsulation / decapsulation; ML-DSA signing / verification; opaque key handles only
TLS	Hybrid TLS 1.3 (classical + ML-KEM group) sample client and server, policy-profile driven (LIGHTRIDER_PQC_TRANSITION)
FIPS	Startup + conditional self-tests, approved-mode status, per-item validation-status reporting (module cert, entropy verification status, PQC status)
EMS (optional)	Telemetry/CBOM synchronization; user-controllable cloud-entropy mix-in; disabled by default with a zero-network-traffic guarantee
Platforms	Windows 11 x86-64 (MSI + Windows service) and Linux x86-64 (deb/rpm + systemd unit)

2.2 Out of scope (V1)

Quantum Light hardware and QPU-derived entropy as *seed* sources; D-QKD; mobile; tactical hardware; customer-specific FIPS operational environments; multi-source entropy pooling; proprietary algorithms;

automatic remediation/migration orchestration in qSearch; macOS. *Do not expand V1 scope without approval.*

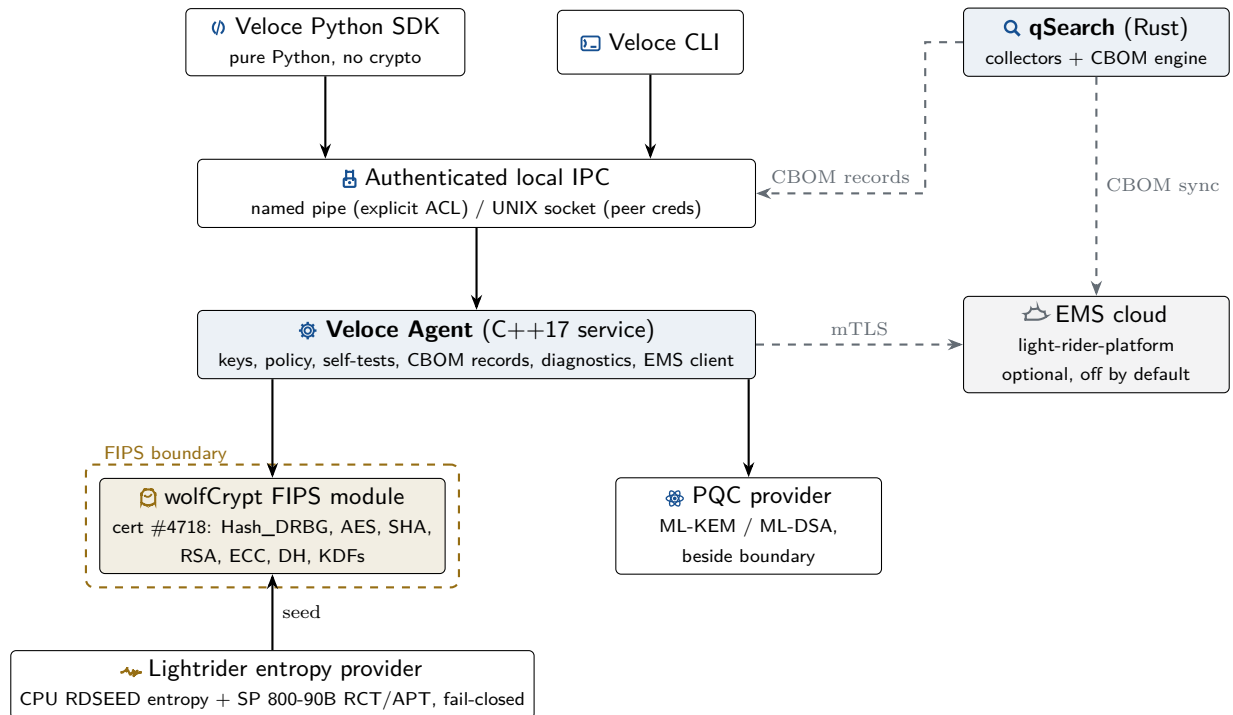
2.3 Standards and policy baseline

Standard	Role	Veloce usage
FIPS 140-3	Module validation	wolfCrypt module v5.2.1, CMVP cert #4718
SP 800-90A	DRBG	Hash_DRBG (SHA-256), 256-bit strength
SP 800-90B	Entropy source health tests	RCT (cutoff 4) + APT (13/512) at $H = 8$ bits/sample on the RDSEED seed stream, 1024-sample startup test (engineering assurance; no ESV certificate exists for v5.2.1, legacy IG 9.3.A applies)
SP 800-90C	RBG construction	RDSEED entropy source seeding the module DRBG directly; the optional OS DRBG source is an RBGC chain and carries no security-strength claim
FIPS 203 / 204	PQC algorithms	ML-KEM-768 (default), ML-DSA
TLS 1.3 (RFC 8446)	Protocol	Hybrid groups, e.g. X25519MLKEM768
CycloneDX CBOM	Inventory format	qSearch + SDK CBOM export
OMB M-23-02	Federal policy	Inventory fields; scanning-first CBOM process
CISA ACDI strategy	Discovery baseline	Coverage model for qSearch collectors

[OPEN: Federal “CBOM minimum elements” are pending; the CBOM schema module must be versioned so it can adopt them without breaking consumers.]

3 System Architecture

One native agent owns all keys, entropy, and FIPS state. Python and the CLI are thin, crypto-free IPC clients. qSearch is a separate Rust binary outside the FIPS boundary that shares only the CBOM data model with the SDK. EMS is reachable only from the agent, only when enabled.



3.1 Component responsibilities

Component	Language	Owns
Veloce Python SDK	Python	Public API, IPC client, typed results; no direct wolfCrypt loading; all cryptography via the agent.
Veloce CLI	Rust	Operator commands: status, self-test, CBOM export, diagnostics, scan.
Veloce Agent	C++17	wolfCrypt lifecycle, entropy, keys (opaque handles), policy profiles, FIPS self-tests, validation status, CBOM records, EMS client, diagnostics.
wolfCrypt module	C	FIPS 140-3 approved algorithms + Hash_DRBG (dynamic library; never statically linked; protected files never modified).
Lightrider entropy provider	C++ (agent)	Custom generate-seed callback: CPU RDSEED entropy acquisition (OS DRBG output only by explicit configuration), SP 800-90B RCT/APT health tests, fail-closed delivery via <code>wc_SetSeed_Cb()</code> ; live entropy-spec reporting including source kind, RBG construction, and strength claim.
PQC provider	C	ML-KEM / ML-DSA from the wolfSSL tree, compiled beside the FIPS boundary (see 5.3).
qSearch	Rust	Collector adapters, normalization, correlation, classification, CBOM generation, risk scoring, reporting. Fully offline-capable.
EMS	cloud	Fleet telemetry, CBOM aggregation, policy distribution, cloud-entropy service.

Repository layout: `agent/`, `python/`, `cli/`, `desktop/` (local web UI: `veloce_desktop.py` + `static/`), `qsearch/`, `ipc/` (versioned, length-prefixed message definitions), `installer/` (MSI + deb/rpm), `tests/` (unit, integration, interoperability, security), `examples/` (tls-client, tls-server), `docs/`, `cbom/`, `vendor/wolfssl/` (licensed source; restricted private repository, excluded from any public repo).

4 Pillar A: qSearch, Cryptographic Search

qSearch discovers cryptography, classifies quantum vulnerability, and maintains the CBOM. Collectors are replaceable adapters; the normalization, correlation, classification, CBOM, risk, and reporting layers are Lightrider-developed. qSearch runs entirely offline, outside the FIPS boundary.

4.1 Collector adapters (replaceable)

Adapter	Backend and method	Platform
Source-code scan	Implemented: recursive Rust walker, case-sensitive literal patterns, selected extensions, 4 MiB file limit; skips common build/vendor directories	Linux; portable source mode planned for Win/macOS
Dependency scan	Planned: package/lockfile + SBOM (SCA) analysis; links findings to packages, container images, and digests	Win + Linux + macOS
Certificate discovery	Implemented: PEM blocks, SHA-256 fingerprints and selected key OIDs; Linux certificate-store collector. Native Windows/macOS stores planned	Linux

Network / PCAP	Planned: Zeek (or Suricata) for live traffic and PCAP: TLS versions, cipher suites, key-exchange groups	Linux native
Windows capture	TShark/Wireshark + Npcap producing PCAP; analysis by Zeek on a Linux sensor, WSL, or analysis server (Zeek on Windows is experimental)	Windows
Endpoint / config	Implemented on Linux: <code>ldconfig</code> , library paths, <code>/proc/crypto</code> , <code>/proc/*/maps</code> , SSH, certificate stores, and policy files; Windows/macOS collectors planned	Linux

License policy: collectors must be Apache-2.0, MIT, or BSD. **[OPEN: GPL-licensed components (verify CryptoScan, Zeek, Suricata licenses) require legal approval of the distribution model before bundling; fallback is “bring-your-own-collector” integration.]**

4.2 Lightrider-owned layers

Normalization → correlation → quantum-vulnerability classification → CBOM generation → risk prioritization → reporting → EMS synchronization. These layers are written in Rust and are collector-agnostic.

4.3 One internal finding format

All collectors emit a single normalized record; downstream layers are tool-agnostic.

Core fields	Asset; application/process; algorithm; cryptographic service (key establishment, encrypted connection, digital signature, authentication, code/firmware signing); protocol; key/parameter length; certificate fingerprint; library/package; version; source of evidence; confidence level; last-observed date
Provenance label	directly observed statically detected vendor reported inferred manually validated
Workbook mapping	Provenance + method map onto the CBOM workbook <i>Discovery Findings</i> sheet (Discovery Method, Finding Confidence, Validation Status columns); see Appendix C.

4.4 Detection targets and outputs

V1 prioritizes *active* cryptography: key establishment, encrypted connections, digital signatures, authentication, code/firmware signing. Classification flags RSA, DH, ECDH, ECDSA (and small-key symmetric use) as quantum-vulnerable; ML-KEM/ML-DSA usage is recognized and reported as PQC-ready.

Outputs: **JSON** (canonical), **CSV**, **CycloneDX CBOM**, **M-23-02 inventory fields**, **executive summary**, and **SDK runtime validation records** (the Veloce agent’s own CBOM self-report merges into the same store).

4.5 Coverage assurance and outlier names

No finite keyword list can guarantee discovery of every crypto wrapper, alias, generated binding, renamed module, or proprietary API. qSearch therefore treats keyword matches as evidence, not proof, and uses a layered coverage model:

1. Curated rules include canonical standards, historical names (for example Kyber and Dilithium), vendor APIs, language bindings, protocol groups, OIDs, imports, package names, and configuration keys. Rules move to a versioned external JSON/TOML file with rule identifier, aliases, classification, risk, source, and regression fixture.

2. Structural collectors examine certificates, dependency manifests, linked/loaded libraries, symbols, processes, and protocol observations. These catch modules whose application-facing name does not contain a known algorithm keyword.
3. Every rule has positive, negative, alias, comment/test, and boundary fixtures. Controlled corpora measure precision, recall, false positives, and explicit blind spots. Unknown providers are reported as context for analyst review, never silently classified as safe.
4. Customer/vendor dictionaries may add organization-specific wrapper and product names without rebuilding qSearch. Signed rule-pack versions are recorded in every run log and CBOM record.

4.6 Optional AI triage

AI is an enrichment stage after deterministic collection; it is never the sole source of a CBOM fact. qSearch batches findings and, only with explicit tenant opt-in, submits the minimum necessary context (rule ID, path token, matched line and a bounded surrounding window) to a Lightrider AI gateway. The gateway may classify active code versus comments, tests, examples, or wrappers; suggest aliases; explain risk; and propose remediation. The original match, file/line evidence, model/version, prompt-policy version, confidence, and human disposition remain in the audit record.

The model-provider credential is stored once in Lightrider's server-side secret manager and is never shipped in qSearch, a wheel, a CLI flag, a repository, or a customer report. Customers authenticate to Lightrider with a scoped tenant credential; the gateway attaches the provider credential and applies quotas, redaction, retention, and audit policy. Calls are batched per scan/chunk, not made once per keyword. A bring-your-own-key mode and a fully local/offline model are optional deployment profiles. AI failure, denial, or timeout must not block deterministic scanning or alter the zero-network guarantee when EMS/AI is disabled.

4.7 Performance approach

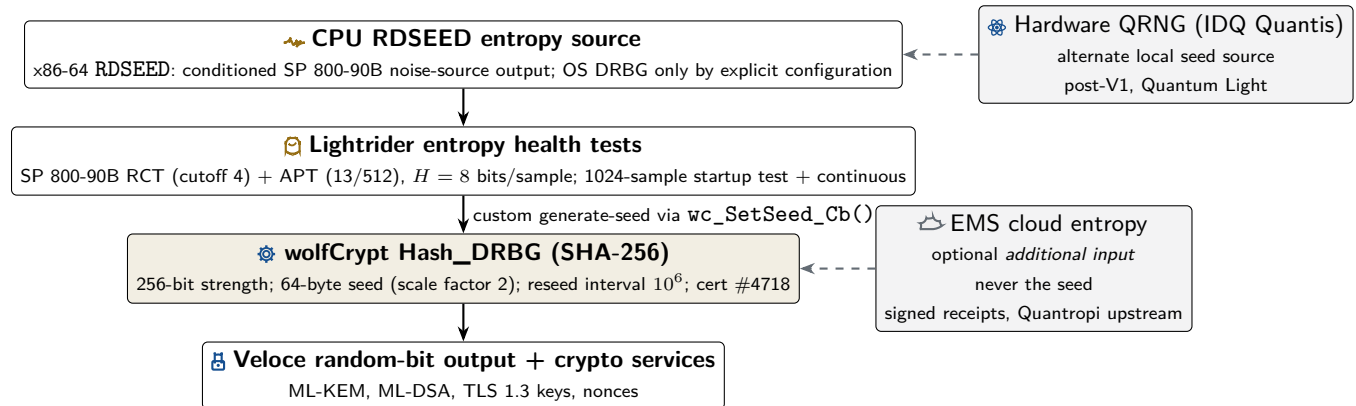
Current implementation: a single-threaded depth-first `read_dir` walk loads each readable file up to 4 MiB as UTF-8, then applies every literal pattern to every line. It does not use an index, AST, memory mapping, Aho–Corasick, or incremental hashes.

Target upgrade: parallel ignore-aware walking; one compiled Aho–Corasick/regex-set pass; bounded streaming; and incremental re-scan keyed on file hashes. AI runs only on the much smaller finding set. Benchmarks and false-positive/blind-spot reporting are release gates (Section 9).

5 Pillar B: Entropy Generator and FIPS Crypto Core

The crypto core consists of a verified local entropy provider feeding a validated DRBG, with PQC and TLS services above it. All failure modes are fail-closed.

5.1 Entropy pipeline



Wiring. The module itself makes no entropy claim (#4718 SP §11.1, vendor-confirmed); its security policy requires the crypto officer to register an external seed source. After loading the module and checking `wolfCrypt_GetStatus_fips()`, the agent registers the *Lightrider generate-seed callback* with `wc_SetSeed_Cb()` — the mechanism wolfSSL’s FIPS FAQ §2 prescribes for custom seeding — and instantiates the FIPS DRBG with `wc_InitRng()`. The callback acquires seed material from the processor’s RDSEED instruction (conditioned output of the CPU’s SP 800-90B noise source, delivered directly to the module DRBG so that no second DRBG sits in the seed path), runs the SP 800-90B RCT/APT health tests over every block, and returns it only on pass; any acquisition or health-test failure returns an error, and because the FIPS build defines `WC_RNG_SEED_CB`, the DRBG then refuses to instantiate or reseed — there is no silent fallback path. `wolfEntropy` is not used: wolfSSL confirmed on 2026-08-27 that it was never tested with module v5.2.1 and recommends against pairing it with the legacy module. The agent exposes `FipsCore::randomBytes()` as a callback to the separate PQC provider: ML-KEM keygen requests 64 bytes; ML-KEM encapsulation, ML-DSA keygen, and hedged ML-DSA signing request 32 bytes each. Applications never supply or retrieve raw entropy.

Seed source selection (revised 2026-09-17). The configuration key `entropy.source` selects `rdseed` (default) or `os-drbg`; there is no runtime fallback between them. `os-drbg` reads `getrandom()` on Linux and `BCryptGenRandom` on Windows, which return the operating system DRBG’s output. Seeding the module DRBG from that output forms an SP 800-90C RBGC construction, and where the operating system RBG is not validated for the operational environment no security strength can be claimed for the chain; the same is true of the module’s portable `wc_GenerateSeed` callback, which reads `/dev/urandom`. The `os-drbg` source therefore exists only for processors without RDSEED (arm64) and for development, and `validation_status()` reports it as `rbg_construction: SP 800-90C RBGC` with `security_strength_claim: none`. The RDSEED source is reported as an SP 800-90B entropy source seeding the DRBG directly; its health-test parameters assume the processor vendor’s full-entropy rating ($H = 8$ bits per 8-bit sample, $\alpha = 2^{-20}$), and a formal entropy assessment or ESV certificate for the RDSEED source on each operational environment remains an open vendor item. When RDSEED is absent and `os-drbg` is not configured, the agent fails closed.

Fail-closed policy. If entropy initialization, a health test (RCT/APT), or a reseed fails, the agent transitions to a degraded state: all key generation, signing, encapsulation, and TLS handshakes are refused; `health()` and the validation-status API report the failure; an event-log entry and (if enabled) an EMS alert are emitted. There is no API that returns raw entropy, and entropy is never sent to EMS.

Certification position (resolved 2026-08-27). No ESV certificate applies to this module and none is claimed: wolfSSL confirmed ESV #E335 belongs to their separate v6.0.0 module (currently “Pending Resubmission”), that `wolfEntropy` was never tested with v5.2.1, and that v5.2.4 — the first version designed for ESV-mode seeding — is about a year from certification as a *new* certificate. Module #4718 was validated before 2026-01-01 and therefore falls under *legacy* IG 9.3.A, which permits an external documented seed source without an attached ESV validation. Veloce’s compliance position is exactly that: the module’s own certificate plus a documented, health-tested external seed source, reported in `validation_status()` as `esv_certified: false, source_kind, rbg_construction, and security_strength_claim: none`. The Lightrider

RCT/APT layer is engineering assurance and is never represented as an ESV credit. Per wolfSSL: “testing is the only confidence when it comes to FIPS”; the verification layer plus the gate battery is that testing.

Entropy providers. V1 ships exactly one seed provider: the Lightrider local entropy provider (CPU RDSEED entropy, health-tested per block; OS DRBG output only by explicit configuration). No external hardware entropy device is integrated in V1. Two quantum-entropy integrations follow post-V1: the **Quantropi** quantum-entropy service (QiSpace SEQR), delivered over the network through the EMS cloud-entropy channel as DRBG additional input (Section 6.2); and the IDQ Quantis QRNG as a local alternate seed source for Quantum Light deployments. `list_entropy_providers()` enumerates providers, so new sources are added without API change.

5.2 FIPS module integration

- Module: wolfCrypt FIPS 140-3 v5.2.1, CMVP cert #4718; library wolfSSL 5.9.2. Built with `--enable-fips=v5` via autotools (CMake is not FIPS-capable in this tree). Loaded as a shared library / DLL, never statically linked; protected module files are never renamed, patched, or recompiled without wolfSSL approval.
- In-core integrity: HMAC-SHA256 check; build flow is `configure → make → ./fips-hash.sh → make → testwolfcrypt` (Appendix A).
- Self-tests: power-on self-test at load plus conditional algorithm self-tests (CASTs) for AES-CBC/GCM, HMAC-SHA-1/2/3, DRBG, RSA sign, ECDSA, ECC-CDH, DH-Z, and the TLS 1.2/1.3/SSH KDFs. The SDK’s `run_fips_self_tests()` calls `wc_RunAllCast_fips()`, and status is read from `wolfCrypt_GetStatus_fips()`.
- Record for every build: DLL/so hash, source version, build settings, compiler, FIPS certificate, entropy provider status, operating environment (feeds the CBOM).
- Windows FIPS build (vendor-confirmed 2026-08-27): **only DLL builds are supported with FIPS.** The bundled README’s static-library statement is a legacy copy/paste error; wolfSSL is issuing a correction (wolfSSL/wolfssl PR #11142). Build `IDE/WIN10/wolfssl-fips.sln` in Visual Studio with configuration `DLL Debug | x64` or `DLL Release | x64`, using the vendor-supplied `v5.2.1 user_settings.h` (`HAVE_FIPS_VERSION_MAJOR 5`, `HAVE_FIPS_VERSION_MINOR 2`, `HAVE_FIPS_VERSION_PATCH 1`). The DLL-only rule matches Veloce’s dynamic-loading design exactly.
- OE status (vendor-confirmed 2026-08-27): CAVP/CMVP testing is *completed, awaiting publication* on certificate #4718 and its security policy for **Windows 11 Pro on Intel Core i7-1260P @ 2.10 GHz (Lenovo ThinkPad T14)** and for **Ubuntu 24.04 on Docker 28.5 over Hyper-V (Xeon Platinum 8272CL, Azure Standard_D2s_v4)**. CAVP validation 36918 records the algorithm testing. A completed-but-unpublished OE must not be represented as current CMVP module/OE approval; `validation_status()` reports the Windows OE as `pending_publication` until the SP update lands.
- Porting beyond the tested OEs: wolfSSL does not issue vendor affirmations. At FIPS 140-3 Level 1, CMVP Management Manual §7.9.2 permits *user affirmation* to a compatible operating system once a matching OS row is on the certificate; the affirmation is keyed to the OS, not the CPU, and the risk sits with the affirmer. wolfSSL recommends retesting whenever CPU or OS changes.
- The CMVP entry carries the caveat “no assurance of the minimum strength of generated SSPs”; entropy assurance is Veloce’s responsibility via the Lightrider entropy provider (Section 5). The wolfCrypt FIPS User Guide / COGM (received 2026-08-27) governs which build settings are permitted; Veloce’s build flags are checked against it at G0. **[VENDOR: Single remaining item: notify when the two tested OEs appear on the published #4718 security policy; only then does the Windows validated-deployment claim switch on.]**

5.3 PQC provider outside the FIPS boundary

The #4718 boundary contains classical algorithms only, and this FIPS build strips the SHAKE/XOF primitives that ML-KEM and ML-DSA require, so **PQC cannot link inside the v5 FIPS build**. V1 therefore compiles the wolfSSL ML-KEM/ML-DSA sources (FIPS 203/204, with fault-hardening flags) as a separate *PQC provider* library beside the boundary, drawing all randomness from the FIPS DRBG via the `*WithRandom` APIs.

- Validation status reports each fact separately (no generic `FIPS=True`): module validated + cert number; `entropy_esv_certified: false` + `entropy_verified_local: true`; PQC algorithm and its validation status; `pqc_inside_fips_boundary: false`; OE; self-test results. Values come from the installed module, never hardcoded.
- **[VENDOR: Track wolfSSL's FIPS v7 "PQ-FS" submission (boundary wrappers for ML-KEM, ML-DSA, LMS, XMSS, SLH-DSA + SHA-512 DRBG). Migrating to v7 flips `pqc_inside_fips_boundary` to true with no API change; the provider interface is designed for that swap.]**
- CAVP/ACVTS testing of the PQC provider (known-answer + interface tests) is planned so the CBOM can cite algorithm-level evidence even before v7. **[OPEN: Choose ACVTS scope and lab.]**

5.4 Hybrid TLS 1.3

Sample client and server negotiate a wolfSSL-supported hybrid group. Candidates: `X25519MLKEM768` (default), `SecP256r1MLKEM768`, `SecP384r1MLKEM1024`. The public API accepts a *policy profile*, never raw algorithm strings:

```
session = veloce.tls.connect(host=..., port=443, policy="LIGHTRIDER_PQC_TRANSITION")
```

[VENDOR: Do not hardcode the production group until wolfSSL confirms the group supported by the licensed bundle, its validation status, the approved build configuration, and interoperability expectations.]

5.5 Performance plan

Performance is verified by measurement. Using `wolfcrypt/benchmark` plus Veloce-level harnesses, each release publishes: DRBG throughput (MB/s, single and multi-threaded via per-thread RNG instances); entropy source fill rate; ML-KEM-768 keygen/encap/decap ops/s; ML-DSA sign/verify ops/s; hybrid TLS 1.3 handshakes/s; IPC round-trip latency. The baseline is captured at stage S1 on the reference environments; release gate = published performance report with no regression >10% against baseline. **[OPEN: Absolute targets to be set from the S1 baseline.]**

6 Pillar C: EMS Cloud Integration

EMS (`light-rider-platform`) is strictly optional. Veloce is fully functional with EMS disabled; this default is enforced by a zero-network-traffic test.

6.1 Modes and configuration

```
ems: { mode: disabled }   or   ems: { mode: enabled, endpoint: https://..., entropy_mixin:
off|on }
```

EMS may receive	Agent health; SDK and module versions; CBOM records; policy identifiers; self-test status; entropy-provider status; non-sensitive operational telemetry
EMS must never receive	Private keys; shared secrets; <i>raw entropy</i> ; DRBG seeds or state; unencrypted customer data; TLS session secrets

Endpoint contract (resolved from `light-rider-platform`). The EMS data plane is the `ems-egress` service: REST on `:7081` (`POST /v1/entropy/request`, 1–65 536 bytes per call; `GET /v1/sources`; `GET /v1/entropy/pools`; `GET /v1/pubkey`; `GET /v1/receipts/:request_id`), a WebSocket stream (`GET /v1/entropy/stream`) with parameters `policy` and `bytes_per_tick` (default 64, clamped 1–4096) on a fixed 100 ms tick, and a QUIC lane (`:7082`, ALPN `1r-qstream`) that already negotiates hybrid `X25519MLKEM768`. Wire schemas are fixed by `proto/entropy.proto` (`EntropyRequest/Response`, `EntropyReceipt`, `HealthSnapshot`, `SourceSelectionPolicy`). Every response carries a *signed receipt* (Ed25519 today; ML-DSA-65 / MASQ-DS paths exist) containing the contributing sources, pool, extractor algorithm, SP 800-90B health snapshot (RCT/APT pass, bit balance, Shannon estimate, chi-square p-value, quality score 0–100), DRBG algorithm and reseed id, and a nanosecond timestamp; `raw_entropy_stored` is always false. Authentication is a per-device Bearer API key

issued by the `ems-admin` control plane; entitlement tiers (Free/Pro/Enterprise) gate the pools (`pool_fastest`, `pool_highest_quality`, `pool_quantum_verified`). The agent is the only Veloce component that opens the connection, verifies the receipt signature against the pinned EMS public key, and enforces freshness before any use. mTLS device enrollment remains the target transport hardening once EMS supports it. Policies accepted by the service are `fastest_available` (Free tier), `highest_quality`, and `quantum_verified`; the receipt signature is computed over the canonical receipt bytes (sorted keys, `signature` removed, compact JSON).

6.2 User-controllable cloud entropy (V1 addition)

EMS may deliver entropy *to* devices; devices never send entropy to EMS.

Design.

1. The agent (when `ems.mode: enabled` and `entropy_mixin: on`) periodically fetches an entropy packet from EMS over HTTPS (`POST /v1/entropy/request`) using the TLS client of the loaded FIPS module, with the ISRG Root X1 anchor pinned (`ems.ca_file` overrides), an optional Bearer API key (the Free tier is keyless on `fastest_available`), and a receipt verified against the pinned service key (`ems.pubkey_hex`; Ed25519 via the PQC provider's verify-only implementation, or ML-DSA-65) after canonicalization identical to the service (sorted keys, signature removed, compact JSON); freshness (`ems.max_age_s`), packet length, health flags, and `raw_entropy_stored` are checked. mTLS device enrollment remains the target transport hardening once EMS supports it.
2. The packet is mixed into the wolfCrypt DRBG as the SP 800-90A *instantiation nonce*: certificate #4718 exports no additional-input or reseed entry point, so the agent re-instantiates the DRBG through `wc_InitRngNonce()`, which draws fresh seed-source entropy through the registered callback and mixes the packet as nonce via `Hash_df`. The packet is cryptographically mixed in but contributes *zero* to the credited entropy claim. The local Lightrider entropy provider remains the sole seed. The FIPS certificate claims are unaffected.
3. User control: enable/disable per config or at runtime via the SDK (`set_entropy_mixin(...)`); state is visible in `health()`, `list_entropy_providers()`, validation status, and the CBOM (`cloud_entropy_mixin: on/off`).
4. Fail-safe: EMS unreachable, stale, or malformed packets $\Rightarrow$ mix-in is skipped and local operation continues untouched; a status flag records the last successful mix-in. Cloud entropy can never block or degrade local crypto.

Post-V1, the same delivery channel carries quantum-sourced entropy: the Quantropi quantum-entropy service (QISpace SEQR) as the EMS upstream source (the platform's `qispace` collector already ingests TQRND in 4096-byte blocks), and Quantum Light QPU-derived entropy.

6.3 Desktop UI: entropy specs and cloud-EMS control (V1 addition)

The Veloce Desktop application (`desktop/`, a local web UI served only on 127.0.0.1 and backed exclusively by the SDK/IPC — it never touches wolfCrypt) gains an **Entropy** page beside the existing security dashboard and qSearch pages. The page has exactly two jobs:

1. **Show the live entropy streaming specs.** One table, populated from `list_entropy_providers()` and `health()`, never hardcoded: provider name and type (`lightrider-local`, OS kernel interface); DRBG (Hash_DRBG SHA-256, 256-bit strength, reseed interval); health-test status and counters (RCT cutoff 4, APT 13/512, blocks verified, failures: always zero or the agent is degraded); source kind, RBG construction, and strength claim; last seed/reseed time; and, when the mix-in is on, the last EMS receipt summary (pool, contributing sources, quality score, signature status, freshness age, bytes per tick). Raw entropy values are never displayed or exposed.
2. **One switch: cloud EMS on/off.** A single toggle wired to `set_entropy_mixin(...)` plus an endpoint/API-key field (stored in the agent config, not the browser). Turning it on never changes the FIPS story — the panel states in fixed text that cloud entropy is additional input with zero credited entropy and that the local provider remains the sole seed. Turning it off is immediate and is the default state.

The toggle state is mirrored in `health()`, `validation_status()`, and the CBOM (`cloud_entropy_mixin: on/off`).

Acceptance (G3): the page renders correct specs with EMS disabled and no network traffic; enabling the toggle starts authenticated fetches and updates the receipt summary; killing EMS mid-session flips the mix-in status to `stale/skipped` while all local crypto continues; the toggle round-trips through config restart.

7 APIs and Deliverables

7.1 Public Python API (17 functions, fixed by the guidance)

Group	Functions
Lifecycle	<code>initialize()</code> , <code>health()</code> , <code>version()</code>
Discovery	<code>list_policy_profiles()</code> , <code>list_crypto_providers()</code> , <code>list_entropy_providers()</code> , <code>validation_status()</code>
ML-KEM	<code>mlkem_generate_keypair()</code> , <code>mlkem_encapsulate()</code> , <code>mlkem_decapsulate()</code>
ML-DSA	<code>mldsa_generate_keypair()</code> , <code>mldsa_sign()</code> , <code>mldsa_verify()</code>
TLS / FIPS	<code>configure_hybrid_tls()</code> , <code>run_fips_self_tests()</code> , <code>approved_mode_status()</code>
Reporting	<code>export_cbom()</code> , <code>generate_diagnostic_bundle()</code>

Private keys never cross the IPC channel: key APIs return a public key plus an opaque `private_key_handle`. The implementation adds `mldsa_generate_keypair()` (required to exercise signing) and the housekeeping operation `release_key()` for agent-side zeroization. V1 also adds `set_entropy_mixin()` / mix-in status inside `list_entropy_providers()`.

7.2 Deliverables checklist

Python wheel (`veloce_pqc-<ver>-py3-none-any.whl`); native agent; wolfCrypt shared library/DLL; qSearch binary; Veloce Desktop application (security dashboard, qSearch, entropy page per Section 6.3); Windows MSI (Authenticode-signed, service registration, named-pipe ACLs, EMS-disabled default config, upgrade/repair/uninstall) and Linux deb/rpm (systemd unit, socket permissions); CLI; API reference; architecture document; security guide; FIPS integration guide; entropy-provider guide; SBOM and CBOM; sample TLS client/server; test vectors; performance report; known-limitations statement; reseller evaluation package; **Lightrider EULA** and **third-party notices file** (wolfSSL attribution + collector licenses; see Section 8.1); branding assets (logo artwork + ASCII banner, Section 7.5).

7.3 Client installation experience

A client must reach a working, verified state in one action per platform, with no compiler, no configuration file, and no cloud dependency:

- **Windows:** one Authenticode-signed MSI installs the agent service (`VelocePqcAgent`), the wolfCrypt FIPS DLL, the PQC provider DLL, the CLI, and the desktop app; the service starts, runs self-tests, and the desktop app opens on the security dashboard. EMS is disabled by default.
- **Linux:** one `deb/rpm` installs the same set with a systemd unit; `veloce status` or the desktop app confirms the state.
- **Python:** `pip install veloce_pqc` for the SDK; it requires a running agent and contains no cryptography.

First run always shows the same three facts, from live status only: FIPS module (cert #4718 + self-test result), entropy (local provider verified, fail-closed), and PQC provider (beside boundary). Enabling EMS cloud support is one action from the desktop app's entropy page (Section 6.3) or one config line; it is never required for any local function. Every install is reproducible from the release manifest: each artifact carries SHA-256 checksums, the SBOM/CBOM, tested OS/CPU, and gate report (Section 7.3), and the build recipes in Appendix A regenerate byte-comparable libraries from the same bundle and flags.

7.4 Platform-separated development and release layout

All operating systems remain in one repository and share protocol, algorithm, report, and conformance-test definitions. OS-specific code is isolated behind transport, library loader, service, system-discovery, and packaging interfaces; platform conditionals must not spread through cryptographic policy code. The target source layout is:

```
platform/linux/   Unix socket/peer credentials, dlopen, systemd, Linux collectors
platform/windows/ named pipe/ACL, LoadLibraryExW, Windows service, Windows collectors
platform/macos/   local IPC/peer authorization, dylib, launchd, Keychain/macOS collectors
core/, ipc/, python/, qsearch/  shared behavior and schemas
installer/{linux,windows,macos}/ platform package inputs
```

The existing Linux implementation remains in `agent/`, `cli/`, and `qsearch/src/system.rs` until the adapters are extracted; empty or planned platform directories do not imply support. Releases are assembled into explicit targets, never a mixed archive:

```
release/<version>/linux/x86_64/  tar/deb/rpm, ELF binaries and .sos
release/<version>/windows/x86_64/ signed MSI, PE binaries and DLLs
release/<version>/macos/{x86_64,arm64}/ signed/notarized package and .dylibs
release/<version>/common/  protocol/docs plus only genuinely portable files
```

Each target carries its own manifest, SHA-256 checksums, SBOM/CBOM, tested OS/CPU, validation claims, known limitations, and gate report. The current `scripts/make_release.sh` produces Linux x86-64 only; Windows and macOS release folders must not be emitted until their gates pass.

7.5 Startup branding

Client-visible entry points display Lightrider branding at startup. The banner's status line is populated from `validation_status()` at runtime, never hardcoded.

```
--\ / / _ _ | | / _ \ / _ _ |
 \ v / | _ | | | ( ) | ( _ | |
  \ / | _ | _ _ \ _ _ / \ _ | _ |
Lightrider Inc -- Veloce PQC SDK v<version>
FIPS 140-3 #4718 | entropy: verified (local) | approved mode: on
```

Surface	Behavior
Veloce CLI	Prints the ASCII banner + status line on interactive startup. Suppressed when stdout is not a TTY, with <code>--quiet</code> or <code>--json</code> , or when <code>VELOCE_NO_BANNER=1</code> ; machine-readable output is never polluted.
Veloce Agent	Writes the banner and status line to its startup log entry: Windows Event Log source on Windows, journald/syslog on Linux.
Python SDK	No output on <code>import</code> (library convention); banner available explicitly via <code>veloce.banner()</code> .
Installers	MSI UI dialogs carry Lightrider logo artwork; deb/rpm post-install prints a one-line branded confirmation with version and FIPS certificate.
Reports	CBOM exports, diagnostic bundles, and the qSearch executive summary carry a product-identity header (name, version, certificates).

Branding assets live in `assets/branding/` (logo SVG/ICO for installers and documentation; `banner.txt` as the single source for the ASCII art).

8 Security and Compliance

Control area	V1 implementation
Key custody	All private keys and sensitive state live in the agent; opaque handles outside; zeroization on release (tested).
IPC hardening	Windows: named pipe <code>\\.\pipe\LightRider.PQC.v1</code> with an explicit ACL (service identity, authorized local user, local admins; deny anonymous/remote/unauthenticated; never the default descriptor). Linux: UNIX domain socket, mode 0660, <code>SO_PEERCREC</code> checks. Versioned length-prefixed protocol; no raw entropy, DRBG state, keys, or plaintext diagnostics on the channel.
Library loading	Windows: absolute path under <code>Program Files</code> , <code>SetDefaultDllDirectories</code> + <code>LoadLibraryExW</code> restricted search; never in <code>System32</code> or global <code>PATH</code> . Linux: absolute <code>dlopen</code> path, no <code>LD_LIBRARY_PATH</code> trust; DLL/so hash verified against the recorded build hash.
Service identity	Windows service <code>VelocePqcAgent</code> under <code>NT AUTHORITY\LocalService</code> ; Linux <code>systemd</code> unit with a dedicated low-privilege user, <code>ProtectSystem=strict</code> . [VENDOR: Confirm service-account choice against wolfSSL operational-environment requirements.]
Certificates reported	FIPS 140-3 cert #4718 (module v5.2.1), bound to its exact OE; entropy reported as <code>entropy_esv_certified: false, entropy_verified_local: true</code> (no ESV certificate exists for this module); surfaced in validation status and CBOM.
Data hygiene	Diagnostic bundles are redaction-tested; CBOM output contains metadata only: never keys, seeds, passwords, tokens, or customer plaintext (mirrors the CBOM workbook rule).
Supply chain	Licensed wolfSSL source confined to a restricted private repo (<code>vendor/</code>); reproducible FIPS builds; signed installers; SBOM per release; qSearch collector licenses vetted (Apache-2.0/MIT/BSD).

8.1 Licensing and distribution model

Lightrider holds a wolfSSL commercial license (Agreement v.01-24 in the bundle); **Veloce ships under Lightrider’s own commercial license (EULA)**, wrapping wolfCrypt as an embedded component. The agreement’s terms impose the following requirements:

Agreement fact	What Veloce must do
§3: distribute the “Combined Program” in <i>object code</i> only	Ship wolfCrypt exclusively as the compiled DLL/.so inside Veloce installers: never source, never as a standalone library. Already matched by the dynamic-library design and the restricted <code>vendor/</code> repo.
§3: license is contingent on the Combined Program <i>not</i> being a cryptography library that competes with the Products	[VENDOR: Confirm with wolfSSL that Veloce (a PQC SDK exposing crypto services through the local agent, not an embeddable crypto library) is the named Program on the Order Form.] Supporting position: Veloce exposes cryptographic services through the local agent (opaque handles), not a linkable crypto library.
§3: per-Developer licensing (anyone who links, compiles, or edits code using the Products)	Count agent/core developers against the Order Form seat count; qSearch and Python SDK developers never touch wolfCrypt and do not count. Track seats in the repo access list.
§4: retain wolfSSL copyright notices; attribution in documentation permitted; no sublicense/transfer	Keep wolfSSL notices in the shipped binaries and a third-party notices file ; docs state Veloce is “built with wolfCrypt (FIPS 140-3 cert #4718)”. The Lightrider EULA licenses <i>Veloce</i> , and must not purport to sublicense wolfSSL itself.
§5: confidentiality (3 years / trade-secret duration)	FIPS source bundle and build configs stay in the restricted private repo; excluded from any public repository and from customer deliveries (including the Customer Operational-Environment Package).
§7: end-user rights survive termination	End-user license continuity is reflected in the Lightrider EULA.
§10: support covers updates to the licensed version, not upgrades	[VENDOR: The FIPS v7 “PQ-FS” migration is an upgrade; budget a new wolfSSL Order Form for it.]
§11: FAR 12.211/12.212, DFARS 252.227-7015 government rights	Mirror the restricted-rights language in the Lightrider EULA; required for the M-23-02 federal customer base.

Layered result: **Lightrider EULA** (Veloce SDK, all Lightrider-owned code) → **wolfSSL commercial license** (embedded wolfCrypt object code, invisible to the customer beyond notices) → **permissive licenses** (Apache-2.0/MIT/BSD qSearch collectors, listed in the notices file). qSearch and the Python SDK contain no wolfSSL code, so Lightrider may license them freely under its own terms.

9 Test Gates and Acceptance

Release is gate-based: V1 does not ship until every gate passes on both platforms.

Gate group	Must pass
FIPS & entropy	wolfCrypt test suite; startup + conditional self-tests; Lightrider seed-callback health tests (RCT/APT pass and induced-failure); seed-source reporting (RDSEED direct or explicitly labelled OS DRBG chain, no over-claim); induced entropy-failure and reseed-failure tests (fail-closed verified: DRBG refuses to instantiate/reseed on callback error); seed-path proof (strace/ETW evidence that seeds flow through the registered callback); DRBG continuous-test behavior.
PQC	ML-KEM keygen/encap/decap and ML-DSA sign/verify known-answer tests; fault-harden builds; negative tests (corrupted ciphertext/signature).
TLS	Hybrid TLS 1.3 client/server interop; negotiated-group reporting; certificate validation; downgrade rejection; fragmented handshakes; unsupported-server behavior; EMS on/off operation.
IPC & platform	Named-pipe/socket authorization; unauthorized-local-user rejection; remote-access rejection; DLL substitution and search-path attacks; key zeroization; diagnostic redaction.
EMS	Zero network traffic when disabled; telemetry allow-list enforcement when enabled; entropy mix-in on/off behavior incl. EMS-unavailable fail-safe.
qSearch	Controlled sample environments containing known RSA, ECDH, ECDSA, DH, ML-KEM, ML-DSA implementations: expected detections found; false positives and blind spots explicitly reported; CBOM accuracy vs. ground truth; CycloneDX schema validation; offline-mode run.
Packaging & perf	MSI install/upgrade/repair/uninstall; deb/rpm equivalents; config preserved across upgrades; performance and memory benchmarks published.

V1 completion criteria (condensed): Python SDK talks only through the agent; wolfCrypt loaded dynamically; the verified Lightrider seed callback seeds the DRBG (proven, not assumed); hybrid TLS works client/server; full local operation with EMS disabled; EMS enable exposes no secrets; entropy specs and the EMS toggle work in the desktop UI; validation status accurate per-item; CBOM export automatic; installers produce securely configured services; all gates green.

10 Execution Guide: What To Do, In Order

Work proceeds as a dependency-ordered sequence of stages, each closed by an objective gate. The two pillars run in parallel after S0. Vendor items (**[VENDOR:]**) are on the critical path and are opened at S0.

Stage	Target	What to do / exit gate
S0	Foundations	Repo + CI skeleton; Linux FIPS build brought up (<code>--enable-fips=v5</code> , flags checked against the UG/COGM, fips-hash flow, <code>testwolfcrypt</code> green); IPC protocol v1 drafted; wolfSSL engagement opened and <i>answered 2026-08-27</i> (Windows DLL-only configuration, v5.2.1 <code>user_settings.h</code> , legacy IG 9.3.A entropy position, no ESV cert for this module, v7 timeline). <i>Gate G0: FIPS module passes self-tests on the Linux reference machine.</i>
S1	Entropy + core	Agent skeleton (Linux daemon + Windows service); Lightrider seed callback wired via <code>wc_SetSeed_Cb</code> (OS kernel entropy + RCT/APT verification); fail-closed state machine; seed-path proof; PQC provider built beside boundary + KATs; validation-status API; performance baseline. <i>Gate G1: entropy pipeline + ML-KEM/ML-DSA pass all KATs and failure-injection tests on both OSes.</i>

S2	qSearch	Rust engine + finding schema; source/dependency/ certificate collectors; Zeek/PCAP pipeline (Linux native, Windows capture); classification + CBOM (JSON/CSV/CycloneDX); sample-environment accuracy runs. <i>Gate G2: qSearch detects all planted crypto in the controlled environments and reports false positives/blind spots. (Parallel to S1.)</i>
S3	TLS + EMS	Hybrid TLS 1.3 samples + interop; Python SDK surface complete; CLI; EMS entropy client against the ems-egress contract (/v1/entropy/request , receipt verification) + telemetry; cloud-entropy mix-in; desktop entropy page (specs + EMS toggle, Section 6.3); zero-traffic verification. <i>Gate G3: TLS + EMS gate groups green.</i>
S4	Harden & ship	Full security test battery (IPC, DLL, zeroization, redaction); packaging (MSI, deb/rpm, wheel); docs set; performance report; known-limitations statement; reseller evaluation package. <i>Gate G4 = V1 release candidate.</i>

10.1 Component-to-repository traceability

Each repository directory implements a specification section and is accepted by an exit gate.

Repository path	Specification	Gate
vendor/wolfssl/	Appendix A build recipes; §8.1 license constraints	G0
ipc/	§3 IPC protocol; §8 channel ACLs	G1
agent/	§5 entropy pipeline, FIPS integration, PQC provider; §8 key custody, loading, service identity	G1
qsearch/	§4 collectors, finding format, outputs	G2
cbom/	§4 output formats; Appendix C field mapping	G2
python/	§7 API surface; §6 EMS controls	G3
cli/	§7 operator commands; §7.5 banner	G3
desktop/	§6.3 entropy specs + EMS toggle; §7.3 first-run experience	G3
examples/	§5 hybrid TLS 1.3 client/server	G3
tests/	§9 gate definitions	G0–G4
installer/	§7 packaging; §8 secure defaults	G4
assets/branding/	§7.5 logo + banner assets	G4
docs/	§7 documentation deliverables	G4
docs/two-server-manual-test.md	Manual GitHub-synchronized two-host ML-DSA/ML-KEM interoperability procedure	G3

Post-V1 roadmap: wolfSSL FIPS v7 migration (PQC inside boundary); IDQ Quantis QRNG as alternate local seed source (Quantum Light); Quantropi QiSpace SEQR entropy as the EMS upstream, delivered over the mix-in channel; QPU-derived cloud entropy; macOS; multi-source entropy pooling; qSearch remediation orchestration.

11 Risks and Open Items

Risk / item	Prob.	Impact	Mitigation
CMVP publication timing: Windows 11/i7-1260P and Azure Linux OE testing is complete but the #4718 certificate/SP update has not landed	M	M	Build and gate the Windows DLL track now (configuration is vendor-confirmed); ship Windows as pending_publication status until the SP lists the OE; re-check the published SP before any validated-deployment claim.
EMS ems-egress stream handler does not compile at platform HEAD (stream.rs lines 59/65 textually corrupted)	certain	L	Platform team fixes before S3 EMS integration; Veloce's request/response path (/v1/entropy/request) is unaffected; mix-in fail-safe covers outages anyway.

PQC outside FIPS boundary until v7	certain	M	Honest per-item status reporting (already the documented design); ACVTS algorithm evidence; v7 migration path built into the provider interface.
PQC implementation defects (cf. CVE-2026-6330, ML-KEM ARM64 ciphertext-comparison flaw)	L-M	H	Stay current with wolfSSL releases; KAT + negative tests in CI; x86-64-only scope in V1 limits exposure.
Hybrid TLS production group unconfirmed	M	M	Policy-profile indirection means the group is config, not API; default X25519MLKEM768 pending confirmation.
wolfSSL §3 compete clause: Veloce could be read as a “cryptography library”	M	H	Paper Veloce as the named Program on the Order Form at S0; agent-service architecture (no linkable crypto API) is the supporting position; legal review of the Lightrider EULA wrap.
Collector licensing (GPL) blocks bundling	M	M	Legal review at S0; adapters keep collectors replaceable; bring-your-own-collector fallback.
OS kernel entropy quality is asserted by the OS, not ESV-certified	certain	M	Honest reporting (entropy_esv_certified: false); legacy IG 9.3.A position on record; Lightrider RCT/APT verification on every block; QRNG + v5.2.4/v6 ESV-backed modules on the roadmap when certified.
DRBG throughput insufficient for high-rate workloads	M	M	Per-thread DRBG instances (reseeds amortized); measure at the S1 baseline; QRNG roadmap as high-rate alternative.
Federal CBOM minimum elements published mid-project	M	L	Versioned CBOM schema module; workbook already flags mandatory update on issuance.
Zeek-on-Windows immaturity	certain	L	Documented capture-and-forward design (TShark/Npcap → Linux/WSL sensor); native Windows analysis revisited post-V1.

A Build Recipes

Linux FIPS module (bundle on hand)

```

cd vendor/wolfssl
./configure --enable-fips=v5
make
./fips-hash.sh  (updates the in-core HMAC-SHA256 verify hash)
make
./wolfcrypt/test/testwolfcrypt  (must report all tests passed)

```

Notes: autotools is the only FIPS-capable build path in this tree; **--enable-fips** also turns on reproducible builds. **--enable-wolfEntropy** is *removed*: wolfSSL confirmed 2026-08-27 that wolfEntropy was never tested with v5.2.1 and is not to be used with the legacy module; seeding is the Lightrider callback registered at runtime. Check the final flag set against the wolfCrypt FIPS User Guide / COGM (on hand). Record DLL/so SHA-256, source version, compiler, and flags into the CBOM store at build time.

PQC provider (beside boundary)

Separate non-FIPS build of the same wolfSSL tree (or standalone compilation of **wc_mlkem.*** and **wc_mldsa.*** with SHAKE enabled), configured with **--enable-mlkem --enable-mldsa** plus the fault-hardening flags **WC_MLKEM_FAULT_HARDEN** and **WC_MLDSA_FAULT_HARDEN**; randomness supplied exclusively from the FIPS DRBG via the ***WithRandom** APIs; distinct library name and symbol namespace so the two builds can never be confused at load time.

Windows (configuration vendor-confirmed 2026-08-27)

Open the wolfSSL-provided IDE/WIN10 FIPS solution (`wolfssl-fips.sln`) in Visual Studio and select build configuration `DLL Debug | x64` or `DLL Release | x64`. Only DLL builds are supported with FIPS; the README's static-library instruction is a legacy copy/paste error being corrected in wolfSSL PR #11142. Use the vendor-supplied v5.2.1 `user_settings.h` (`HAVE_FIPS_VERSION_MAJOR 5`, `HAVE_FIPS_VERSION_MINOR 2`, `HAVE_FIPS_VERSION_PATCH 1`) and the documented in-core integrity linker settings, then run the same hash-verify-record flow (`fips-hash` equivalent + `testwolfcrypt`) and record the DLL SHA-256. Implement the named-pipe transport, restricted DLL loading (`LoadLibraryExW`), Windows service, Windows collectors, and MSI behind the platform adapters. Seeding uses the same Lightrider callback with `BCryptGenRandom` as the kernel source. The Windows 11 Pro/i7-1260P OE is tested and awaiting publication on #4718; until the published security policy lists it, Windows ships with `validation_status()` reporting `pending_publication` and no validated-deployment claim.

B Manual Two-Host Interoperability Test

The executable step-by-step procedure is maintained in `docs/two-server-manual-test.md`. It assigns 168.231.75.64 (`srv811871.hstgr.cloud`) as signer/ML-KEM sender and 93.127.215.63 (`srv862034.hstgr.cloud`) as verifier/ML-KEM recipient. Both hosts clone the same commit from the private GitHub repository using independent read-only deploy keys; the licensed wolfSSL bundle is provisioned separately and must never enter GitHub. Each host builds, gates, and runs its own local agent. Public keys, ML-DSA signature bundles, and ML-KEM ciphertext move through SSH/SCP; private handles and shared secrets never leave their owning host. The test compares shared-secret SHA-256 fingerprints and records commit, OS/CPU, validation, and gate evidence. It does not claim to implement authenticated payload encryption or hybrid TLS.

C CBOM Field Mapping (SDK → workbook)

The Veloce agent's self-report and qSearch findings both flow into the Light Rider CBOM workbook schema:

Veloce report field	CBOM Inventory column
Crypto library / module + version	Crypto Library/Provider; Library Version; Cryptographic Module
FIPS certificate (#4718)	FIPS 140-3 Certificate
Entropy provider + verification status	Entropy Source; ESV Status/Certificate (recorded as "none / locally verified")
Algorithm + parameter set (e.g. ML-KEM-768)	Algorithm; Key/Parameter Size; Target PQC Algorithm
TLS version + hybrid group	Protocol/Version; Cipher Suite
Approved-mode + self-test status	Notes / runtime validation record
OS edition, build, CPU arch	Operating System (Supplier sheet); Environment
qSearch finding provenance + confidence	Discovery Findings: Discovery Method, Finding Confidence, Validation Status, False Positive?, Blind Spot/Limitation
Scan runs	Scanning Log sheet (tool/method, findings counts, false positives)

Additional SDK-only CBOM fields (JSON/CSV export): agent version, wolfCrypt DLL/so name + SHA-256, EMS enabled/disabled, cloud-entropy mix-in state, installation and last-verified timestamps.

D References

- OMB Memorandum M-23-02, *Migrating to Post-Quantum Cryptography* (Nov 2022).
- CISA, *Strategy for Migrating to Automated Post-Quantum Cryptography Discovery and Inventory Tools* (Sep 2024).
- CISA, *2026 Minimum Elements for a Software Bill of Materials* (Jul 2026).
- NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA); FIPS 140-3; SP 800-90A/B.
- NIST CMVP validated-modules search (wolfSSL):
<https://csrc.nist.gov/projects/cryptographic-module-validation-program>.

6. NIST CMVP certificate #4718 and security policy (checked 2026-08-10):
<https://csrc.nist.gov/projects/cryptographic-module-validation-program/certificate/4718>.
7. NIST CAVP wolfCrypt product/A4308 records, including the Windows algorithm-test environments; CAVP evidence is not CMVP OE approval:
<https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/details?product=16876>.
8. wolfSSL FIPS status page, “Pending Operating Environments currently in testing” (checked 2026-08-10): <https://www.wolfssl.com/license/fips/>.
9. CryptoScan: <https://github.com/csnip/cryptoscan>; Zeek packet analysis: <https://docs.zeek.org>.
10. wolfSSL 5.9.2 release (Jun 23, 2026), commercial FIPS bundle `linuxv5.2.1`; wolfSSL FIPS-FAQ v2.3 (§2: `wc_SetSeed_Cb` and custom generate-seed).
11. wolfSSL written response to Lightrider, 2026-08-27: Windows DLL-only FIPS builds (README correction in PR #11142); v5.2.1 `user_settings.h`; Windows 11 Pro/i7-1260P and Azure Ubuntu 24.04 OEs tested, awaiting #4718 publication (CAVP validation 36918); legacy IG 9.3.A applicability; wolfEntropy not tested with v5.2.1; ESV #E335 = v6.0.0 module; wolfCrypt FIPS User Guide / COGM provided.
12. CMVP Management Manual §7.9 (porting and user affirmation at Level 1, keyed to the operating system).
13. Lightrider EMS platform (`light-rider-platform`): `proto/entropy.proto` wire schemas; `ems-egress` REST/WebSocket/QUIC entropy endpoints and signed receipts.
14. Minimus, *From SBOM to CBOM* and *How to Use CBOMs in Containerized Environments* (industry perspective).
15. Light Rider CBOM Starter Template (`Light_Rider_CBOM_Template_Updated.xlsx`).